

Where we are, and what happens next

Troll Farm status, 10 August 2026

Supersedes the five-day summary of 8 August. No prior knowledge assumed.

One paragraph. We are trying to make a program win more games of an online strategy contest. The competition program has not changed in a week and is not expected to change soon. Everything since has gone into the equipment we use to judge changes, because we discovered that equipment was giving meaningless answers. That work is close to finished for one component and just beginning for another. We also found, along the way, that **the best program we have ever measured is not the one we are running**.

1. Current state

Item	State
Competition program	Unchanged. Scores 22.81 , rank 32 of 137.
Best program we have measured	A different one - 24.76, rank 21 - and it is not running . One measurement only.
Quality checkpoint	Still not usable. Third repair delivered, awaiting review.
Messaging between agents	Working. Two broken messages remain, both known.
Automated tests	110 passing on the coordination tooling; 148 on the repaired harness.
Live competition entry	Untouched. Nothing submitted in over a week.

The bot we are not running

It is called **readable_no_orchard**. Three things are true of it at once, and no other program we have has any two of them: it is the **only human-readable** one we ever submitted (1,475 lines; every other is a single line of 55,000+ characters), it is the **smallest** by actual code, and it holds the **highest score we have ever measured**. We replaced it with something 17% larger and two points worse.

The reason for caution is real, and the tooling raises it unprompted: that 24.76 is a **single run**, and a near-identical program scored 23.27 on its own single run. A 1.5-point spread is wider than the noise we normally assume. One more run would settle it.

2. Why the quality checkpoint still is not usable

The checkpoint runs practice games and reports whether a change is safe. Two independent defects were found in it, and both are the same shape: **the instrument was measuring something other than what it claimed**.

- **A check that flagged correct behaviour as a bug.** It fired 196 times in a test where the thing it detects was mathematically impossible. It was not measuring the delay it was named for; it was measuring an ordinary, intended action.
- **A referee that silently ignored a command.** The practice-game referee accepted the instruction to hire a worker, discarded it, and carried on. The program then re-issued it every turn for the rest of the game. Those games scored as our **cleanest** results - nine checks and a liveness monitor, all silent.

Repairs are on their third round. Rounds one and two were rejected by adversarial review. Round three closes all eleven outstanding objections, passes 148 tests, and survives ten deliberate sabotage mutations with none escaping. **It awaits one acceptance review.** Until that lands, no judgement about any program change means anything.

3. The oscillation work, and what it taught

One character can get stuck pacing between two squares, doing nothing, for up to 97% of a game. You asked for it removed - not for score, but to reduce technical debt and regain control and understanding of the program.

Three agents answered the same question independently. **The fix all three would have endorsed does not work:** in all twenty never-resolving cases the blocking partner never moves, so stopping the pacing produces a stalled character instead of an unstuck one. The warning goes off; nothing improves.

That single measurement redirected the repair, and we have it because one agent checked something the brief never asked about.

Since then the picture has sharpened again, in a way worth knowing:

- The load-bearing fix is now believed to be an **idle-yield rule** - make the stationary idle character move - rather than re-routing the blocked one.
- **But that belief is not yet confirmed.** The finding came from a library built on a *different* program than the one specified, and an independent attempt could not verify it from stored data. The two programs behave identically on this bug, so it probably transfers - but probably is not verified.

4. Making the program's intentions readable

You observed that the program's behaviour is defined by assigning numeric weights to actions, that this hides what the characters are actually trying to do, and that it has already caused you and the agents to disagree about what the program does.

Both reviewers confirmed the problem and corrected the premise: **weights are only about a third of the decision.** The rest is mode selection, filtering, pair compatibility, forced replacement and post-decision movement rewriting. So a simple table of "intention to number" was demoted - it would document the easy third.

The agreed first deliverable is a **Decision Packet:** a generated, complete explanation of one turn - every option considered, why each was rejected, what the program was trying to

do, the numbers and their attainable ranges, and what the movement layer changed afterwards. Its specification is finished.

An audit of the numbers found **10 boundary crossings, 3 ranking inversions and 3 pieces of dead scoring code**. The most striking: one intention is priced at 187 on turn 250 and 7,000 on turn 251 - a jump of up to 961x at an arbitrary turn number.

5. Proposed next moves

Ordered by what unblocks the most. Items 1 and 2 need nobody's decision; items 5 and 6 need yours.

#	Move	Why now	Needs
1	Accept or reject the third harness repair	Nothing can be measured until this lands. It is the only thing blocking the entire quality pipeline.	chatgpt_1
2	Rebuild the oscillation library on the specified program	The existing one is built on a different program. Everything downstream would inherit that.	claude_1
3	Confirm or refute the idle-blocker finding	The whole repair has been redirected around it and it is unverified.	either agent
4	Judge each frozen situation independently	Asks whether a decision was <i>correct</i> . Everything we own today asks only whether it oscillated.	both, split
5	Re-run readable__no_orchard for a second measurement	It may be our best program. One run cannot tell a good program from a lucky draw. Costs nothing but time and uses none of the broken equipment.	your decision
6	Check the repository branch-protection settings	One agent found an attack that could permanently break every agent's messaging with a single push, and declined to test it because the proof is the damage.	only you can

Item 5 is the one I would press.

It is the only move on the list that could improve our competitive standing, it requires no code and no working checkpoint, and it resolves the most valuable open question we have: whether the readable program is genuinely two points better than what we are running, or whether that number was luck. Everything else on this page is equipment.

Item 6 takes a minute and is the only item nobody else can do. It is the last unresolved *critical* finding from the security review of our own messaging system.

6. How much to trust this report

Across the last three days the three agents have corrected each other, and themselves, repeatedly. That is the system working, but it should calibrate how you read any single claim.

Pattern	Count over 3 days
Claims by the coordinator later found unsupported	9

- caught by another agent	4
- caught by the coordinator before publishing	3
- caught by a tool before publishing	2
Agents correcting their own published work unprompted	4 occasions
Findings reached independently by two or more agents	5

The trend is the useful part: early in the week errors were caught by people after publication; by the end they were caught by tools and by authors before it. The single most valuable habit has been requiring two independent reviews before any conclusion is adopted - it has overturned a recommendation or a premise every single time it has been applied.

What has not improved: the score. It has been 22.81 for a week and no move on this page except item 5 would change it. That is the deliberate consequence of deciding to repair the measuring equipment first, and it is worth restating plainly rather than burying.

Prepared 10 August 2026 by local_claude_1. Figures re-derived from committed artifacts and live tool runs at the time of writing. Supersedes 2026-08-08-five-day-plain-english-summary.pdf.